

ExRaBitQ-Disk: Controlling Payload I/O in Quantized Graph Search

Anonymous Author(s)

ABSTRACT

Graph-based approximate nearest neighbor search must balance retrieval accuracy, memory consumption, and storage access when an index exceeds its DRAM budget. Compact vector codes reduce representation size, but a candidate requiring only a small distance computation can still trigger a full disk-page read. We present ExRaBitQ-Disk, a quantized graph search system that uses a resident one-bit representation to decide whether a candidate warrants access to its disk-resident payload. The system constructs a Vamana graph using symmetric four-bit distance estimates and performs online search in three stages: one-bit candidate screening, four-bit traversal scoring, and residual-corrected reranking. Screening and traversal share a query representation and a partial inner product; page coalescing and bounded query-local reuse translate surviving candidates into physical reads. We analyze the approximation errors of construction and querying, and distinguish candidate filtering from page savings. Our evaluation protocol separates fixed-candidate quantizer measurements, controlled graph and storage ablations, and complete-system comparisons under a common memory budget. **[E1: Insert audited, recall-matched throughput and physical-I/O results across the admitted datasets.]** The design treats quantized representations as a means of controlling payload access as well as reducing arithmetic and storage costs.

KEYWORDS

Approximate nearest neighbor search, vector quantization, graph indexes, disk I/O

1 INTRODUCTION

Approximate nearest neighbor (ANN) search is a core operation in vector retrieval. Given a query embedding, the system identifies nearby database vectors without exhaustively evaluating the collection. Graph indexes offer an effective accuracy-latency trade-off, but their adjacency lists and vector representations can exceed an affordable DRAM footprint. DiskANN established a practical approach to this setting by combining a navigable graph with memory-resident compressed representations and disk access [8]. A central systems question is how to spend limited memory so that graph traversal makes useful progress with fewer physical reads.

Quantization addresses part of this problem by representing vectors with short codes. Product quantization (PQ), locally adaptive vector quantization (LVQ), RaBitQ, and its multi-bit extension provide different trade-offs between representation cost and distance estimation [1, 3, 4, 6]. Nevertheless, code length is only an indirect measure of disk-search cost. Several candidate records may share a page, a record may require page padding, and a page may be revisited during the same query. Reducing arithmetic per candidate

does not remove a read that has already been issued. Conversely, filtering a candidate saves no physical transfer if another surviving candidate requires the same page.

Recent work motivates a closer connection between representations and execution. SymphonyQG integrates RaBitQ, graph layout, and FastScan for in-memory search [5]. SAQ improves quantization through code adjustment and dimension segmentation, and also supports progressive distance estimation [7]. These contributions make clear that low-bit computation and staged estimation are established techniques. The question addressed here is more specific: how should a staged estimator govern access to graph-search payloads when those payloads reside on disk?

ExRaBitQ-Disk places that decision before the payload read. Each vector has a four-bit primary code and a separately encoded residual correction. A resident copy of the primary code's most significant bit plane, together with scalar factors, provides a cheap screening estimate. Only candidates that pass this screening request their full payload. Their four-bit distances guide traversal; the surviving search pool is subsequently reranked with a floating-point query and residual correction. The graph itself is constructed from symmetric comparisons of primary codes. Thus, the design connects the representation used for construction to the representations used for screening and traversal, while retaining distinct estimators for each task.

This organization creates three technical challenges. First, symmetric code-to-code construction introduces errors at both endpoints; it cannot inherit a single-endpoint estimator's guarantee without further analysis. Second, screening must preserve enough promising candidates despite query quantization and an approximate search threshold. Third, logical filtering must reduce physical page demand after packing, batching, and caching. We address these challenges with an explicit estimator hierarchy, conservative handling of invalid screening estimates, and a page-based read path that preserves candidate processing order.

The contributions are as follows:

- **An integrated disk-search design.** ExRaBitQ-Disk combines symmetric four-bit Vamana construction with one-bit screening before payload access, four-bit traversal, and residual-corrected reranking. It builds on existing quantization and graph primitives rather than claiming a new general-purpose quantizer.
- **An explicit error and resource model.** We separate construction error, query quantization error, final scoring error, and search omissions. We also account for resident side information, residual codes, page padding, and physical reads, which are hidden by a nominal bit rate.
- **A layered evaluation.** Fixed-candidate measurements isolate scoring behavior; fixed-native-graph ablations examine storage mechanisms; complete-system experiments assess

the recall-throughput trade-off. [E1: Replace this evaluation statement with the principal measured finding once the formal test artifacts pass the stated protocol.]

2 PROBLEM AND DESIGN RATIONALE

2.1 Search objective

Let $X = \{x_1, \dots, x_N\} \subset \mathbb{R}^D$ be the database and $q \in \mathbb{R}^D$ a query. We use squared Euclidean distance $d^2(q, x) = \|q - x\|_2^2$. For a returned set $A_k(q)$ and exact ground-truth set $G_k(q)$, retrieval quality is

$$\text{Recall}@k = \frac{1}{|Q|} \sum_{q \in Q} \frac{|A_k(q) \cap G_k(q)|}{k}. \quad (1)$$

The index consists of a directed graph, encoded payloads, and routing metadata. Given a search-index memory budget M and page size P , the objective is to maximize measured throughput at a target recall ρ , subject to accounted search memory not exceeding M . Search memory includes resident codes, model data, all worker workspaces, and caches. Process peak resident set size (RSS) is reported separately. The evaluation uses $k = 10$ and $P = 4096$ bytes. Cosine retrieval is represented by squared L2 only when both database and query normalization have been verified.

2.2 From candidates to page demand

Let S_q be the candidates whose payloads are requested, including the entry point and candidates admitted by screening and $\pi(x)$ the physical pages containing a candidate's payload. Ignoring evictions, the payload pages needed by one query form the union

$$\mathcal{P}_q = \bigcup_{x \in S_q} \pi(x). \quad (2)$$

With a finite cache, actual transfers depend on the sequence of requests. If $\mathcal{B}_t$ is the set of payload pages requested by batch t and C_t the pages resident immediately before that batch, then

$$V_{\text{payload}}(q) = P \sum_t |\mathcal{B}_t \setminus C_t|. \quad (3)$$

Here a batch includes any pages spanning a large record, and the cache state changes after each access. Graph reads are counted separately and added to total bytes. Coalescing adjacent missing pages reduces request count without necessarily reducing transferred bytes. Query-local reuse can reduce both when a requested page remains cached.

The value of a gate therefore depends on which pages its rejected candidates would have required. As an illustrative case, suppose three records share a page. Rejecting two records avoids no read if the third survives. Rejecting all three can avoid that page. This distinction motivates reporting candidates, unique pages, transferred pages, and I/O requests separately.

2.3 Precision should follow the access decision

The resident one-bit layer is intended to reject unpromising candidates before their larger representation is fetched. The primary code then provides a more detailed score for the admitted survivor set. Finally, residual correction concentrates additional arithmetic

on the search pool. These are stages of computation, not necessarily three physical transfers: the reference layout stores primary and residual records together, so a primary-payload read also transfers residual bytes. This cost is included throughout the paper.

3 EXRABITQ-DISK

3.1 Overview and notation

Offline construction computes a shared center, transforms and encodes database vectors, constructs a quantized Vamana graph, and exports adjacency and payload pages. Online search prepares the query once, screens newly discovered neighbors, scores admitted candidates, and reranks the final pool. The main configuration uses one center, ordinary four-bit residual coding, and no adaptive projection or experimental locality layout.

Symbol	Meaning
N, D, d	Database size, input dimension, padded encoding dimension
c, T	Shared center and padded randomized Hadamard transform
z_x, y	Transformed centered database vector and query
n_x, n_q	Norms of the original centered vectors
h_x, λ_x	Centered primary-code coordinates and ratio scale
b_x, ℓ_x	Primary code's one-bit MSB and remaining three-bit labels
$\hat{e}_x$	Decoded residual correction
R, L_b, α	Graph degree cap, construction search width, pruning parameter
L, B, K_r	Query pool capacity, frontier beam, rerank cap
ε, τ	Screening tolerance and current pool-tail score

3.2 Primary representation and residual correction

Centering and transformation. We compute c from a deterministic database training sample. Queries do not participate in training. Define $r_x = x - c$, $r_q = q - c$, $n_x = \|r_x\|_2$, and $n_q = \|r_q\|_2$. Vectors are zero-padded to d , the smallest power of two not below the input dimension rounded up to a multiple of 64. A shared random-sign Hadamard transform gives $z_x = Tr_x$ and $y = Tr_q$. In exact arithmetic $T^T T = I_D$, hence

$$d^2(q, x) = n_q^2 + n_x^2 - 2y^T z_x. \quad (4)$$

Preserving the original norms permits distance estimation by approximating only the cross term. Padding is part of the physical representation: for example, $D = 960$ becomes $d = 1024$, and $D = 1536$ becomes $d = 2048$.

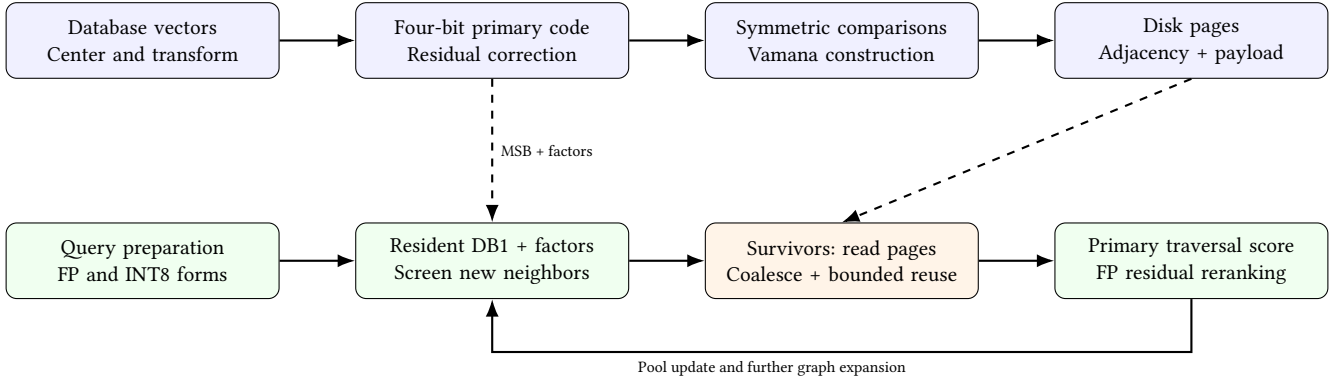


Figure 1: ExRaBitQ-Disk overview. Offline encoding supports symmetric graph construction and a resident screening sidecar. Online screening precedes candidate payload reads. In the ordinary layout, residual bytes accompany the primary record, although their correction is evaluated at final reranking. The dashed paths show index components used during querying; all resident metadata and caches are charged to the memory budget.

Four-bit primary code. For $n_x > 0$, let $u_x = z_x/n_x$. Following the multi-bit RaBitQ representation [3], the primary code stores labels $k_{xi} \in \{0, \dots, 15\}$ and represents the direction by $h_{xi} = k_{xi} - 7.5$. The implementation searches amplitude thresholds to find a well-aligned signed half-integer code. It does not perform PQ codebook clustering or SAQ’s coordinate adjustment. Nor do we assume this finite search finds the global optimum over all discrete codes.

For a valid nondegenerate code, define

$$\begin{aligned} \lambda_x &= \frac{n_x}{u_x^\top h_x}, \\ \tilde{z}_x &= \lambda_x h_x, \\ \hat{d}_4^2(q, x) &= n_q^2 + n_x^2 - 2y^\top \tilde{z}_x. \end{aligned} \quad (5)$$

The stored primary scale includes the factor two in the cross term. The ratio scale has the useful identity $z_x^\top (\tilde{z}_x - z_x) = 0$: its reconstruction error is orthogonal to the database direction in exact arithmetic. This identity is not a guarantee of exact distance estimates for arbitrary queries. Zero norms and invalid scale factors use explicit fallback branches.

Residual correction. The residual is $e_x = z_x - \tilde{z}_x$. Each block of 16 coordinates uses an MSE-selected scale a_j , stored in FP16, and signed four-bit labels

$$v_{xi} = \text{clip}(\text{round}(e_{xi}/a_j), -7, 7), \quad \hat{e}_{xi} = a_j v_{xi}. \quad (6)$$

Labels are computed using the stored-and-decoded scale. The final score is

$$\hat{d}_{4+\text{res4}}^2(q, x) = n_q^2 + n_x^2 - 2y^\top (\tilde{z}_x + \hat{e}_x). \quad (7)$$

The original n_x^2 remains in this expression. Replacing it with the squared norm of the reconstructed vector would define a different estimator. Residual reranking is approximate scoring from compressed data; it is not exact reranking from raw vectors.

3.3 Symmetric quantized graph construction

We construct a Vamana graph using the primary codes at both distance endpoints. For valid vectors x and v , the implemented same-center comparison is

$$\hat{d}_{\text{sym}}^2(x, v) = n_x^2 + n_v^2 - 2n_x n_v \text{clip}\left(\frac{h_x^\top h_v}{(u_x^\top h_x)(u_v^\top h_v)}, -1, 1\right). \quad (8)$$

The query side of a construction comparison is prepared from an encoded database vector, allowing its code and factors to be reused across comparisons. Symmetry follows from exchanging the two endpoints. Clipping keeps the estimated directional inner product in its feasible range; it does not establish a triangle inequality or identical decisions to a floating-point graph.

Vamana candidate discovery and robust pruning use this score in place of raw-vector distance [8]. For a node x , construction searches for candidate neighbors, selects a nearby candidate v , and removes candidates rendered redundant by that selection. In squared-distance notation, the pruning comparison has the form $\alpha_t \hat{d}_{\text{sym}}^2(v, w) \leq \hat{d}_{\text{sym}}^2(x, w)$, subject to the implementation’s candidate cap and degree limit R . The implementation increases the active factor α_t from one to the configured α over pruning passes, with increments capped at 1.2; previously occluded candidates can be reconsidered at a later pass. The exact construction parameters and graph hash accompany each measured index.

This construction reduces dependence on raw vectors during graph distance comparisons and uses the same primary representation as traversal. It does not make offline and online scores identical: online traversal has an uncompressed or INT8 query endpoint, whereas construction quantizes both endpoints. Its benefit must therefore be tested through construction cost and subsequent navigation quality. We make no claim that symmetric quantization is necessary for all successful graph indexes.

3.4 One-bit screening before payload access

Query preparation. The query retains y and its original norm and also creates an INT8 approximation $\hat{y}$. The ordinary path uses one query-wide scale and reproducible dither:

$$\begin{aligned} s_q &= \max_i |y_i|/128, \\ Q_i &= \text{clip}(\lfloor y_i/s_q + U_i \rfloor, -128, 127), \\ \hat{y}_i &= s_q Q_i. \end{aligned} \quad (9)$$

Here $U_i \in [0, 1)$ is deterministically seeded pseudorandom dither. Saturation and a fixed seed prevent us from asserting unconditional unbiasedness of every implemented query score. A zero query uses a protected path.

Resident side information. Split $k_{xi} = 8b_{xi} + \ell_{xi}$, where $b_{xi} \in \{0, 1\}$ and $\ell_{xi} \in \{0, \dots, 7\}$. The resident DB1 sidecar stores b_x and screening factors. Its masked accumulation computes

$$s_x = \sum_i \hat{y}_i (b_{xi} - 1/2). \quad (10)$$

For $\gamma_x = u_x^\top \text{sign}(u_x)/\sqrt{d}$, the nondegenerate factors are

$$\begin{aligned} A_x &= \frac{4n_x}{\sum_i |u_{xi}|}, \\ E_x &= \frac{2n_x}{\sqrt{d-1}} \sqrt{\frac{1 - \gamma_x^2}{\gamma_x^2}}. \end{aligned} \quad (11)$$

The gate forms the screening score

$$g_x = n_x^2 + n_q^2 - \text{clip}(A_x s_x + \varepsilon E_x n_q, -2n_x n_q, 2n_x n_q). \quad (12)$$

This expression follows the form of a RaBitQ uncertainty allowance [4]. We call it a screening score because the implemented transform, INT8 query, and adaptive search threshold have not been shown to satisfy a deterministic lower-bound guarantee. Larger ε makes this score more permissive for a fixed candidate and threshold.

Admission rule. Search maintains a sorted pool of capacity $L \geq k$ and current tail score τ . A newly discovered candidate is rejected before reading its payload only if the pool is full, its screening estimate is valid, and $g_x > \tau$. Otherwise it proceeds to full primary scoring. Marking newly discovered nodes visited prevents repeated evaluation through another edge; it also means that a rejected node is not automatically reconsidered later. Gate errors can therefore affect the search path.

3.5 Traversal scoring and final reranking

The primary representation decomposes as $h_{xi} = 8(b_{xi} - 1/2) + (\ell_{xi} - 3.5)$. For candidates admitted through a valid gate, traversal reuses s_x and computes only the remaining primary-code contribution:

$$\hat{d}_{4, \text{INT8}}^2(q, x) = n_q^2 + n_x^2 - 2\lambda_x \left(8s_x + \sum_i \hat{y}_i (\ell_{xi} - 3.5) \right). \quad (13)$$

The entry point and invalid-gate candidates instead use the floating-point primary score. Final reranking also recomputes the primary score from the floating-point query and adds residual

correction. The query-preparation object is shared across these stages, but the final residual inner product does not use the INT8 query.

Algorithm 1 describes the reference query path. Each frontier is selected from the current pool; its nodes are processed sequentially. A frontier beam is not an asynchronous graph-I/O queue depth. Admitted candidates are processed in batches of at most 64. Page sorting does not change the input candidate order for distance evaluation or pool updates.

Algorithm 1: ExRaBitQ-Disk query, ordinary layout

Input: q , graph G , capacity $L \geq k$, beam $B > 0$, rerank cap Kr

- 1 Prepare floating-point y , INT8 approximation $yhat$, and query norm.
- 2 Reset the query-local page cache and visited state.
- 3 Read entry node 0's payload; initialize the pool with its FP primary score.
- 4 While the pool contains an unexpanded candidate:
 - 5 Select up to B leading unexpanded candidates; mark them expanded.
 - 6 For each selected node, in order:
 - 7 Read its adjacency; collect fresh neighbors and mark them visited.
 - 8 Compute DB1 screening estimates for the fresh neighbors.
 - 9 Reject only valid estimates above the tail of a full pool.
 - 10 For each survivor batch of at most 64 candidates:
 - 11 Map IDs to payload pages; deduplicate and identify cache misses.
 - 12 Coalesce adjacent misses, read, and restore candidate order.
 - 13 Score all candidates, reusing the DB1 inner product when valid.
 - 14 Recheck the gate against the updated pool tail; insert survivors.
 - 15 Select $\min(\max(Kr, k), \text{pool size})$ leading candidates.
 - 16 Obtain their payloads and score with FP query plus residual correction.
 - 17 Return the k candidates with the smallest corrected scores.

The second gate in line 14 occurs after the batch read and distance computation. It can avoid pool insertions, but cannot be counted as avoided payload I/O. Similarly, final reranking only corrects scores inside its selected set; it cannot recover neighbors lost during traversal.

3.6 Physical layout and memory accounting

The ordinary layout stores adjacency pages separately from payload pages. A payload record contains the primary code and its metadata followed by the residual record. The resident DB1 copy duplicates a bit plane already present in the full primary code; the disk representation is not just the remaining three bits.

Component	Bytes per vector in the ordinary layout
Resident DB1 bit plane	$d/8$
Resident screening factors	20 in the audited ABI
Compact primary record	$d/2 + 17$
Residual record, including FP16 block scales	$d/2 + d/8 + 8$
Combined disk payload before page padding	$9d/8 + 25$

For a payload size $S \leq P$, each page stores $r = \lfloor P/S \rfloor$ whole records, and logical ID i maps to byte offset $\lfloor i/r \rfloor P + (i \bmod r)S$. Larger records occupy an integral number of consecutive pages. With $d = 1024$, $S = 1177$ bytes: three records fit in a page and 565 bytes remain unused. The payload alone therefore exceeds nine effective bits per original dimension when $D = d$, before page padding, DB1 duplication, or graph storage. We use “four-bit” to identify the primary code, not the total system budget.

The direct read path sorts and deduplicates batch page IDs and merges consecutive missing pages where supported. A bounded query-local LRU cache shares its byte budget between graph and payload pages, using the file identity to distinguish equal page numbers. Entries are reset between queries. The reference setting allocates 4 MiB per worker for this cache, including its metadata; 32 workers therefore require approximately 128 MiB before other workspaces. Capacity misses and evictions can cause repeated reads, so reuse does not guarantee a page is read only once for an entire query.

Under the experimental budget policy, mandatory resident structures and all worker workspaces are accounted first; any optional cross-query page cache must fit in the remaining budget. The C0 policy prohibits cross-query caching but permits bounded within-query reuse. A smaller budget cannot be satisfied by silently leaving an unaccounted full code array resident. If mandatory structures do not fit, an implemented and verified streaming fallback is required; otherwise that operating point is reported infeasible.

4 ANALYSIS

4.1 What symmetric construction preserves

Let $a_x = \tilde{z}_x - z_x$ and $a_v = \tilde{z}_v - z_v$. Before directional clipping, expansion of the symmetric cross term gives

$$\hat{d}_{\text{sym,raw}}^2 - d^2(x, v) = -2(z_x^\top a_v + z_v^\top a_x + a_x^\top a_v). \quad (14)$$

Consequently,

$$|\hat{d}_{\text{sym}}^2 - d^2(x, v)| \leq 2(n_x \|a_v\|_2 + n_v \|a_x\|_2 + \|a_x\|_2 \|a_v\|_2). \quad (15)$$

This deterministic, reconstruction-dependent bound follows from Cauchy–Schwarz. Clipping does not increase the error because the true inner product lies in the clipped interval. The single-vector orthogonality identity does not cancel the cross-vector terms. In particular, it does not prove that pruning decisions or graph connectivity match a floating-point construction.

A local pruning decision is preserved when its margin exceeds its possible score error. Write the exact pruning margin as $m =$

$d^2(x, w) - \alpha d^2(v, w)$ and let pairwise score errors be bounded by η_{xw} and η_{vw} . If $|m| > \eta_{xw} + \alpha \eta_{vw}$, the exact and approximate margin have the same sign. This is a sufficient condition for an individual comparison on a fixed candidate set. Candidate discovery can itself change under approximate distances, so it is not a theorem of identical final graphs. The evaluation must measure both index construction and search quality.

4.2 Scoring error and search omission

For final reranking, define $t_x = \tilde{z}_x + \hat{e}_x$. In exact arithmetic,

$$\begin{aligned} \hat{d}_{4+\text{res}4}^2 - d^2(q, x) &= -2y^\top (t_x - z_x), \\ |\hat{d}_{4+\text{res}4}^2 - d^2(q, x)| &\leq 2n_q \|t_x - z_x\|_2. \end{aligned} \quad (16)$$

The score error depends on alignment between the query and the remaining reconstruction error. Lower reconstruction MSE alone does not imply a strict recall improvement for every query. During INT8 traversal, an additional term appears:

$$\hat{d}_{4,\text{INT}8}^2 - d^2(q, x) = -2y^\top (\tilde{z}_x - z_x) - 2(\hat{y} - y)^\top \tilde{z}_x. \quad (17)$$

Floating-point reranking removes the second term for its selected candidates. It does not repair omissions caused by the graph, gate, finite pool, or rerank cap. Moreover, the gate compares against approximate pool scores. A pointwise probabilistic error statement for a base estimator would not by itself prove loss-less pruning over this adaptive search. We therefore evaluate the gate empirically and do not claim exact search, zero false pruning, or a new distribution-free recall guarantee.

4.3 Computational and I/O costs

Let F_q be the number of fresh candidates, S_q the number scored with the primary code, and K_q the rerank count. Transforming a query takes $O(d \log d)$ time. Scalar work for screening, primary scoring, and reranking is $O((F_q + S_q + K_q)d)$, with distinct constants and SIMD kernels. The sorted candidate pool can require $O(L)$ work per insertion; the current visited array also incurs $O(N)$ initialization per query. These terms matter as the database and worker count grow.

The gate adds resident storage proportional to $N(d/8 + 20)$ and adds work for every fresh candidate. It is useful when that work is outweighed by avoided payload transfers and scoring. Its selectivity can fall at high recall or with a large pool, and page sharing can dilute candidate-level savings. The design therefore predicts a measurable trade-off rather than universal acceleration. We test that trade-off with recall-matched comparisons and physical-I/O counters.

5 EXPERIMENTAL METHODOLOGY

5.1 Questions and measurement protocol

The evaluation asks three questions: does the primary representation provide useful accuracy per physical byte; which screening and storage mechanisms reduce page demand; and how does the complete system compare at a matched recall? We use the three experimental layers defined below. **This draft specifies the evaluation protocol; quantitative results remain pending the artifact checks identified in Section 6.**

Primary disk experiments use 32 query workers, 4 KiB aligned direct I/O, and a 2 GiB search-index DRAM budget. The direct asynchronous readers use a configured maximum of 128 in-flight operations; the scope of that limit must be recorded with the run. DiskANN uses its native `io_uring` path, while the research disk ports use `libaio`. We report these as distinct backends. Direct I/O bypasses the operating-system page cache; it does not establish that the storage controller is cold.

Queries are deterministically divided into disjoint validation and test sets, with nonempty splits and immutable input and query-order hashes. Validation selects parameters and the measurement policy; formal plots use independent test measurements from a pinned run. The experiment seed is 20260813; the representation’s sampling and transform seed is separately recorded, with 100 in the reference implementation. Throughput timing starts after all workers complete initialization and warm-up and ends at completion of measured work, before teardown. **[E2: Insert the measured machine, device/controller, filesystem, compiler/SIMD configuration, warm-up procedure, test counts, and measurement durations from the selected run.]**

Each operating point has one formal measurement, `repeat_id = 0`. QPS is completed test queries divided by measured wall time. We report `Recall@10`, QPS, and query-level p50 and p95 latency; p99 is diagnostic when test counts are insufficient. Query-latency quantiles are not across-run uncertainty estimates. No confidence interval or multiple-run median is inferred from a single measurement.

5.2 Datasets

Dataset	Database vectors	D	Evaluation role
AGNews	769,382	1024	Core: text embeddings
DBpedia	990,000	1536	Core: high-dimensional text embeddings
GIST	1,000,000	960	Core: visual descriptors
UCI SIFT10M	10,000,000	128	Scale extension; independently constructed query split
Deep1B subset	9,990,000	96	Scale extension; not the full billion-vector collection
MSMARCO	113,520,750	1024	Scale extension
BIGANN10M	10,000,000	128	Scale extension; distinct from UCI SIFT10M
Cohere10M	10,000,000	768	Scale extension

These are the planned data sizes, not a claim that all eight evaluations are complete. An extension enters the results only after its inputs, exact ground truth, candidate sets, quantizer artifacts, graphs, and complete-system indexes pass validation. The UCI SIFT10M split uses source rows `[0, 10,000,000)` for the database and `[11,154,866, 11,164,866)` for queries; the intervening rows are unused. Its squared-L2 ground truth must be computed against that database and cannot be borrowed from BIGANN. Dataset source identifiers, preprocessing, normalization checks, actual query counts, and hashes are retained with the artifacts. **[E2: Verify dataset release and embedding-model identifiers before final submission.]**

5.3 Fixed-candidate quantizer comparison: 05A

We compare PQ_4bit, SQ_4bit, SAQ_B4, and the ExRaBitQ primary-code configuration on identical candidate IDs and ordering. The primary setting places payloads on disk; a resident mode provides a separate computation reference outside the disk experiment’s memory cap. Candidate width ranges from 10 to 30 in steps of one, 40 to 100 in steps of ten, and 140 to 580 in steps of 40.

The experiment reports fixed-candidate recall, mean and p95 relative distance error, pairwise ordering flips, QPS, effective bits per input dimension, bytes transferred, and read amplification. Nominal four-bit settings do not establish equal physical storage: codebooks, factors, dimension padding, and any auxiliary codes must be included. Primary-only accuracy must be distinguished from

residual-corrected accuracy. **[E3: Pin the exact 05A scoring end-point and physical record format for each exported artifact.]** The fixed candidate set bounds this experiment’s recall, so it is not an end-to-end graph-search comparison.

5.4 Controlled graph and storage ablations: 05B

PQ, SQ, and SAQ use the same floating-point Vamana graph, verified by its hash. ExRaBitQ-Disk uses its own symmetric-quantization graph. All ExRaBitQ-Disk storage ablations share that native graph. Thus, cross-method curves include differences in construction and search semantics; only the within-family controls isolate a fixed graph.

The planned sequence is full-four-bit resident without a gate, resident DB1 with disk payloads, DB1 plus coalescing, and DB1 plus coalescing and query-local reuse. The first configuration is a compute reference and is admissible under the main budget only if its complete resident state fits. It changes both residency and, in the current implementation, the query-scoring path. Its difference from the gated disk path cannot be attributed solely to the gate. A clean gate-effect estimate additionally requires matching payload residency and scoring precision while toggling only admission; this control is identified as a required attribution experiment rather than an existing result.

We record DB1 checks and survivors, primary candidates and transferred pages, graph expansions, actual distance evaluations, requests, bytes, and p95 latency. Primary scoring counters include the entire computed batch before the second gate, while reranking is reported separately. Coalescing and reuse comparisons retain the graph, query order, estimator, gate tolerance, and search settings. A symmetric-versus-floating-point construction comparison should likewise hold the query pipeline fixed to test construction independently. **[E4: Supply these matched controls and the four planned ablation curves.]**

5.5 Complete-system comparison: 05C

We compare ExRaBitQ-Disk with DiskANN-PQ-Disk and research disk ports of SymphonyQG, OG-LVQ, and Glass-NSG. DiskANN uses an official native disk path. The other ports preserve their upstream graph and quantization cores but are not presented as official disk implementations of those projects. Implementation parity and their read scheduling are audited before admission. In particular, SymphonyQG neighbor scores use the current node’s edge-associated payload; reading an undecided candidate’s whole row would change the evaluated search behavior.

Each system uses its own index and search-strength parameter. The planned width sweep is 1–30, 40–100 in steps of ten, and 140–580 in steps of 40, subject to each method’s valid range. ExRaBitQ-Disk requires $L \geq 10$ for top-10 search; unsupported settings are excluded, and historical clamped settings must not appear as distinct operating points.

Main figures compare only identical budget and cache policies. At a target recall, we report the highest-throughput measured point satisfying that target, together with its actual recall, latency, and I/O values. All measured points remain available; we do not divide maximum QPS at one width by bytes averaged across other widths. An unreached recall is reported as such. Construction,

quantizer training/encoding, disk export, index size, accounted search memory, and measured peak RSS are reported with distinct scopes.

5.6 Memory sensitivity and reproducibility

In addition to the 2 GiB main setting, the protocol includes C0 and at least three budgeted settings spanning mandatory resident structures through useful page-cache capacity. A budget comparison uses the same accounting rules, optional cache cap, LRU policy, warm-up, and query order across systems; algorithm-specific mandatory memory is reported explicitly. Each figure identifies its policy. These runs assess whether the gate’s memory cost remains worthwhile as payload pages become cacheable. **[E5: Fill the feasible budget values and sensitivity results after complete memory accounting.]**

Formal results must be traceable to input, graph, index, and binary hashes, implementation fingerprints, independent test traces, and a single immutable run identifier. The primary evidence must reside in the current published disk-result directories. A filename containing “formal” is insufficient if the actual worker count, timer scope, memory accounting, or parity checks do not meet this protocol.

6 RESULTS AND EVIDENCE STATUS

The current measurements cover AGNews and GIST with five systems each, and DBpedia with ExRaBitQ-Disk and the SymphonyQG disk port. The frozen test snapshot contains 543 search-width settings across 12 completed artifacts. Each setting uses 32 workers, C0 without cross-query caching, 4 KiB pages, and one measured run following 100 warm-up queries. AGNews and GIST use 800 test queries per setting; DBpedia uses 9,000. The evaluated ExRaBitQ-Disk variant uses a locality layout and therefore does not directly validate the ordinary layout described in Section 3. Formal acceptance remains pending: the verified 2 GiB process address-space limit does not replace complete memory-category accounting, and timing and storage-cache comparability still require review.

6.1 End-to-end recall, throughput, and I/O

The high-recall throughput, read-volume, and tail-latency curves expose a throughput–transfer trade-off. At a minimum Recall@10 of 0.95, the highest-throughput qualifying ExRaBitQ-Disk setting achieves 537.52 queries/s on AGNews at recall 0.9569, compared with 179.93 queries/s for DiskANN-PQ at recall 0.9608. On GIST, the corresponding values are 109.30 queries/s at recall 0.9514 and 31.76 queries/s at recall 0.9571. These observed throughput ratios are 2.99 and 3.44, respectively. They compare a common recall requirement rather than identical attained recall, and do not estimate variability across repeated runs.

At those same selected settings, ExRaBitQ-Disk reads approximately 6.45 times as many bytes per query on AGNews and 6.42 times as many on GIST as DiskANN-PQ. Its corresponding p95 latencies are 81.43 and 400.72 ms, compared with 258.93 and 1,175.69 ms for DiskANN-PQ. Thus, the measured throughput advantage cannot be explained as a reduction in transferred bytes relative

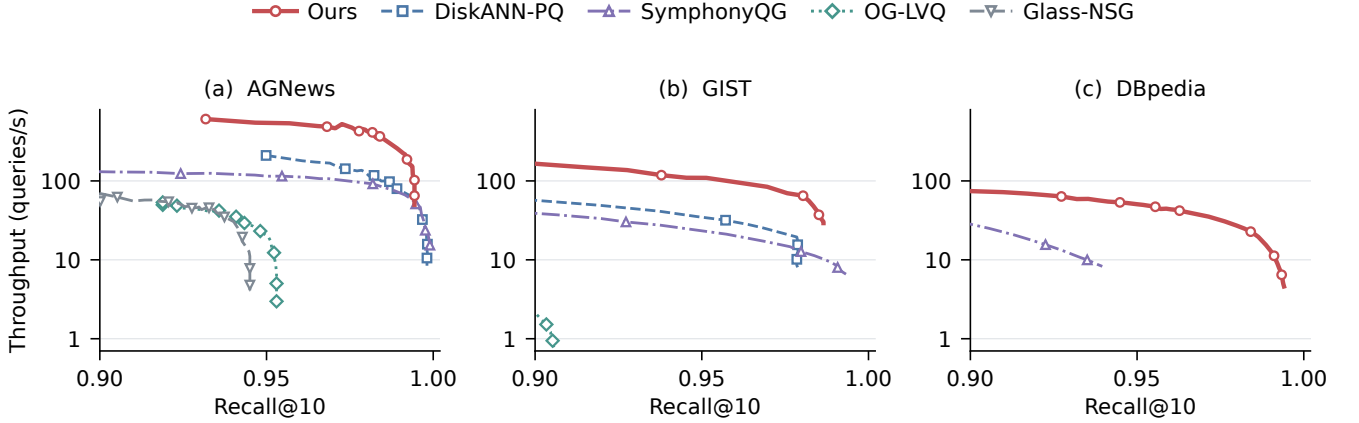


Figure 2: Throughput at high recall. Each curve retains every measured search-width setting; markers appear every fifth setting for readability. All panels use the same logarithmic vertical scale. Ours denotes ExRaBitQ-Disk with locality layout; SymphonyQG, OG-LVQ, and Glass-NSG denote disk ports. DBpedia has only two completed systems. Measurements use 32 workers and C0 (no cross-query caching), with one run per setting; no between-run uncertainty is estimated.

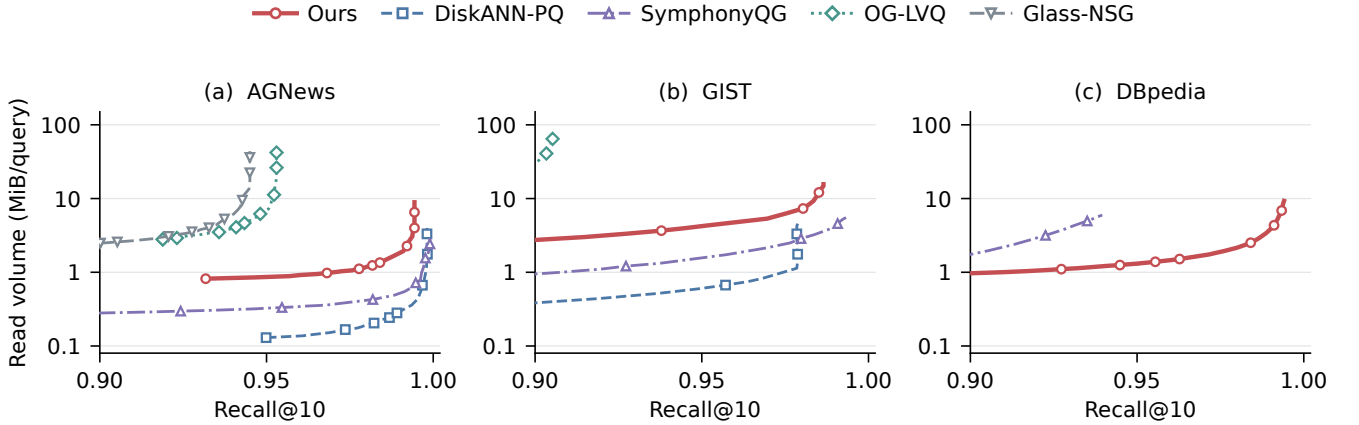


Figure 3: Read volume at high recall. Mean bytes read per query are expressed in MiB, with identical logarithmic vertical scales across datasets. The method identities and missing-system scope are those of Figure 2. Higher throughput need not imply fewer transferred bytes. Measurements use 32 workers and C0 (no cross-query caching), with one run per setting; no between-run uncertainty is estimated.

to DiskANN-PQ; a causal explanation requires the controlled ablations and timing definitions specified in Section 5.

On DBpedia, ExRaBitQ-Disk reaches the 0.95 target at 49.68 queries/s and attained recall 0.9518, whereas the completed SymphonyQG disk-port sweep reaches a maximum recall of 0.9396. This provides no qualifying throughput ratio at that target. Missing systems are left unmeasured. The high-recall views use the same 0.90–1.00 window and per-metric vertical scales across datasets; the full sweep and all source rows remain available with the figure artifacts. [E1: Complete formal acceptance, remaining datasets, and repeated measurements before making a general performance claim.]

6.2 Quantization quality and mechanism attribution

The screening counters show that fewer candidates survive DB1 than are checked, while primary payload reads dominate the plotted page counts at larger widths. These descriptive measurements distinguish candidate screening from physical page demand; they do not measure the counterfactual reads without screening.

[E3–E4: Insert the fixed-candidate accuracy/storage figure and fixed-native-graph ablation figure.] This evidence must

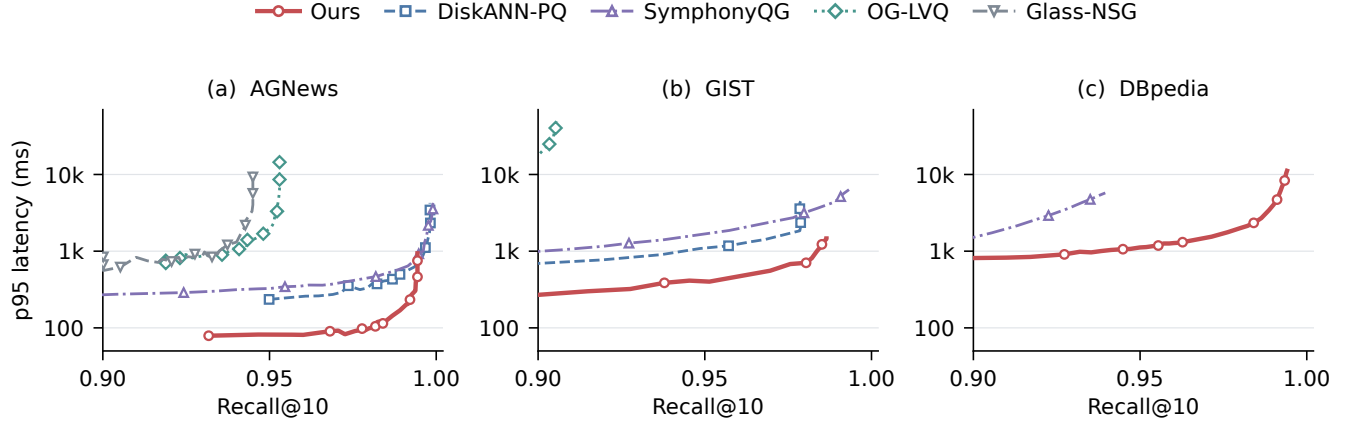


Figure 4: Tail latency at high recall. The 95th percentile is computed over test queries within each setting, not over repeated runs. Vertical scales are logarithmic and identical across datasets. Method identities follow Figure 2. Measurements use 32 workers and C0 (no cross-query caching), with one run per setting; no between-run uncertainty is estimated.

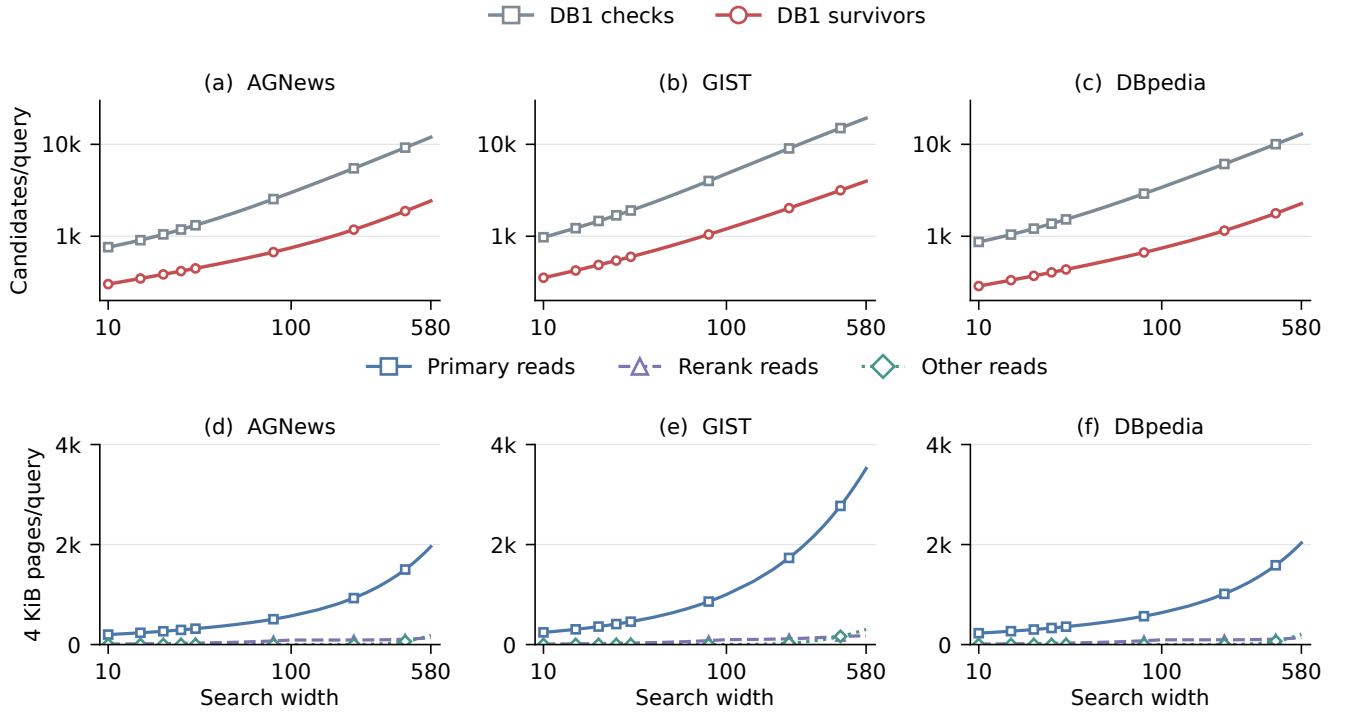


Figure 5: Screening and page demand in the locality variant. Top: mean DB1 checks and survivors per query, on logarithmic axes. Bottom: mean primary, reranking, and other 4 KiB read pages per query, on a linear vertical scale and logarithmic search-width axis. Other reads equal total read sectors minus primary and reranking pages. These counters do not isolate a causal screening benefit. Measurements use 32 workers and C0 (no cross-query caching), with one run per setting; no between-run uncertainty is estimated.

distinguish primary-code accuracy from residual reranking, candidate rejection from avoided pages, and avoided pages from coalesced requests. The mechanism claim requires showing that a gate

reduces physical reads at an acceptable recall cost under a matched

control. A lower survivor count alone is insufficient. The graph-construction comparison must also establish whether encoded construction changes downstream recall or required search width.

6.3 Construction, deployment, and memory sensitivity

[E2, E5: Insert construction/export and memory-sensitivity results.] Report from-scratch costs consistently; reused baseline graphs must not be charged as newly constructed in one comparison and treated as free in another. Show when the mandatory state fits and when optional caching changes the trade-off. A timing breakdown is interpretable only with nonoverlapping definitions: historical fields named I/O wait may include page lookup, allocation, copying, or decoding, and cannot alone establish a pure storage-wait fraction.

7 RELATED WORK

Graph-based and disk ANN. DiskANN combines Vamana with an SSD-oriented search system [8]. NSG develops a sparse navigable graph for efficient search [2]. ExRaBitQ-Disk adopts graph traversal and Vamana-style pruning; its focus is the placement of a screening stage before disk payload access. The research Glass-NSG and other ports are evaluated as concrete implementations under the stated storage protocol, with no implication that their disk behavior is guaranteed by the original papers.

Vector quantization. PQ composes independently quantized subspaces [6]. LVQ uses per-vector scaling with scalar quantization and targets compressed graph-search efficiency [1]. RaBitQ introduces randomized low-bit distance estimation with error analysis, and Extended RaBitQ expands the bit-rate range [3, 4]. These are foundations rather than claims of the present work. The representation here uses a uniform four-bit primary code, a derived one-bit screen, and residual correction, with their combined storage explicitly charged.

Joint and progressive designs. SymphonyQG couples graph structure, edge-local quantized layouts, and FastScan, including implicit reranking during in-memory traversal [5]. SAQ uses code adjustment and PCA-based segmentation with a bit allocation plan, and offers multi-stage estimation [7]. ExRaBitQ-Disk instead uses a shared-center representation and exposes candidate admission as a disk-payload decision. This distinction motivates the evaluation; it does not establish that progressive estimation or quantization-graph integration is itself new.

8 DISCUSSION AND CONCLUSION

ExRaBitQ-Disk organizes compressed graph search around a concrete resource: payload pages requested by a query. Its resident one-bit screen precedes payload access, its primary code supports both construction and traversal, and residual correction refines the retained candidate set. The analysis explains why precision, candidate count, and page transfers must be treated separately.

The design has identifiable limits. The gate occupies memory and may reject useful routes; its implemented score is not a verified lossless lower bound. Symmetric construction can change both candidate discovery and robust pruning. High-recall searches may

admit many candidates, and a cache large enough to retain frequently accessed payloads may reduce the gate’s value. The reference layout transfers residual data with primary data, while experimental split-residual locality and adaptive-routing branches require separate evaluation. The present formulation concerns static squared-L2 search and does not establish update, filtered-search, or arbitrary inner-product behavior.

The resulting systems hypothesis is that a low-cost resident representation can trade modest memory and screening work for fewer payload transfers without an unacceptable recall loss. Establishing the size and scope of that benefit requires the controlled and complete-system results specified above. **[E1: Replace this final evidence-pending sentence with the bounded, measured conclusion once the formal evaluation is complete.]**

REFERENCES

- [1] Cecilia Aguerrebere, Ishwar Bhati, Mark Hildebrand, Mariano Tepper, and Ted Willke. 2023. Similarity search in the blink of an eye with compressed indices. <https://doi.org/10.48550/arXiv.2304.04759> arXiv:2304.04759
- [2] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2017. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. <https://doi.org/10.48550/arXiv.1707.00143> arXiv:1707.00143
- [3] Jianyang Gao, Yutong Gou, Yuexuan Xu, Yongyi Yang, Cheng Long, and Raymond Chi-Wing Wong. 2024. Practical and Asymptotically Optimal Quantization of High-Dimensional Vectors in Euclidean Space for Approximate Nearest Neighbor Search. <https://doi.org/10.48550/arXiv.2409.09913> arXiv:2409.09913 [cs.DB]
- [4] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. <https://doi.org/10.48550/arXiv.2405.12497> arXiv:2405.12497 [cs.DB]
- [5] Yutong Gou, Jianyang Gao, Yuexuan Xu, and Cheng Long. 2024. SymphonyQG: Towards Symphonious Integration of Quantization and Graph for Approximate Nearest Neighbor Search. <https://doi.org/10.48550/arXiv.2411.12229> arXiv:2411.12229 [cs.DB] Preprint version consulted; project identifies the work as SIGMOD 2025.
- [6] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. <https://doi.org/10.1109/TPAMI.2010.57>
- [7] Hui Li, Shiyuan Deng, Xiao Yan, Xiangyu Zhi, and James Cheng. 2025. SAQ: Pushing the Limits of Vector Quantization through Code Adjustment and Dimension Segmentation. <https://doi.org/10.48550/arXiv.2509.12086> arXiv:2509.12086 [cs.DB] Version 1 consulted; local upstream README identifies the work as SIGMOD 2026.
- [8] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, Vol. 32. <https://proceedings.neurips.cc/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html>